

UNIT 2

Software Testing Strategies and Techniques

- **Types of Testing Techniques- Introduction to Black Box, White Box, Grey Box**

Software testing techniques are generally categorised based on whether the internal structure and implementation details of the software are known to the tester. The three primary types are Black Box, White Box, and Grey Box testing.

Black Box Testing

Introduction:

Black Box testing is a testing methodology where the internal structure, design, and implementation of the item being tested are **unknown** to the tester. This method focuses on the functionality of the application based purely on the requirements and specifications. The tester interacts with the system's external interfaces by providing inputs and examining the outputs, without any knowledge of how the system processes those inputs internally.

Key Characteristics:

- **Focus:** Functionality and user experience.
- **Knowledge Required:** None of the internal code or structure.
- **Perspective:** End-user perspective.

Techniques & Examples:

- **Equivalence Partitioning:** Dividing inputs into valid and invalid groups and selecting test cases from each.
- **Boundary Value Analysis:** Testing values at the extreme ends of valid input ranges.
- **Decision Table Testing:** Testing complex business rules represented in a table format.

White Box Testing

Introduction:

White Box testing (also known as clear box, glass box, or structural testing) is a method of testing where the internal structure, design, and implementation details are **known** to the tester. This approach requires the tester to have programming skills to examine the code, internal paths, and infrastructure. The goal is to verify the internal workings of an application rather than just its functionality.

Key Characteristics:

- **Focus:** Code structure, internal logic, and implementation.
- **Knowledge Required:** In-depth knowledge of source code.
- **Perspective:** Developer perspective.

Techniques & Examples:

- **Statement Coverage:** Ensuring every line of code is executed at least once.
- **Decision (Branch) Coverage:** Ensuring every branch (e.g., if/else statements) is executed .
- **Path Testing:** Testing all possible paths through a given function or procedure.

Grey Box Testing

Introduction:

Grey Box testing is a strategy that combines elements of both black box and white box testing. In grey box testing, the tester has **partial knowledge** of the internal structure or access to design documents and the database, but does not have complete access to the source code . This approach allows for more targeted testing while still maintaining a degree of separation between the developer and the tester.

Key Characteristics:

- **Focus:** Functionality and internal structure, with limited information.
- **Knowledge Required:** Partial knowledge (e.g., access to database schemas or design documents).
- **Perspective:** Hybrid of end-user and developer.

Techniques & Examples:

- **Matrix Testing:** Testing variables to ensure proper system performance.
- **Regression Testing:** Testing the impact of new code changes on existing functionality using some structural knowledge.
- **Pattern Testing:** Analyzing past software defects to identify potential current vulnerabilities.

Difference between Black Box and White and Grey Box Testing

1.	This testing has Low granularity.	This testing has a medium level of granularity.	This testing has high-level granularity.
2.	It is done by end-users and also by the tester and developers.	It is done by end-users (called user acceptance testing) and also by	It is generally done by testers and developers.

		testers and developers.	
3.	Here, Internals are not required to be known.	Here, Internals relevant to the testing are known.	Here, the Internal code of the application and database is known.
4.	It is likely to be less exhaustive than the other two.	It is kind of in-between.	Most exhaustive among all three.
5.	It is based on requirements, and test cases on the functional specifications, as the	It provides better variety/depth in test cases on account of high-level knowledge	It can exercise code with a relevant variety of data.

	internals are not known.	of the internals.	
6.	If algorithm testing is not suited best for that.	If used algorithm testing is also not suited best for that.	If algorithm testing is, it is suited best for that.
7.	It is suited for functional or business testing.	It is suited for functional or business domain testing deeply.	It is used for all.
8.	This testing involves validating the outputs for given inputs, the application	Herein, we have a better variety of inputs and the ability to extract	It involves structural testing and enables logic coverage,

9.	being tested as a black-box technique. This is also called Opaque-box testing, Closed-box testing, input-output testing, Data-driven testing, Behavioral, Functional testing	test results from the database for comparison with expected results. This is also called translucent box testing	decisions, etc. within the code. This is also called Glass-box testing, Clear-box testing, Design-based testing, Logic-based testing, Structural testing, and Code-
----	--	--	---

10 .	Black-box test design techniques- • Decision table testing • All-pairs testing • Equivalence partitioning • Error guessing	Gray box test design techniques- • Matrix testing • Regression testing • Pattern testing • Orthogonal Array Testing	based testing. White-box test design techniques- • Control flow testing • Data flow testing • Branch testing
11 .	Black Box testing provides resilience and security against viral attacks.	Gray Box testing does not provide resilience and security against	White Box testing does not provide resilience and security

		viral attacks.	against viral attacks.
--	--	-----------------------	-------------------------------

Classification of Testing- Functional testing and non-Functional testing

What is Functional Testing?

Functional testing is a type of software testing in which the system is tested against the functional requirements and specifications. Functional testing ensures that the requirements or specifications are properly satisfied by the application. This type of testing is particularly concerned with the result of processing. It focuses on simulation of actual system usage but does not develop any system structure assumptions. It is basically defined as a type of testing which verifies that each function of the software application works in conformance with the requirement and specification. This testing is not concerned about the source code of the application. Each functionality of the software application is tested by providing appropriate test input, expecting the output and comparing the actual output with the expected output.

What is Non-functional Testing?

Non-functional testing is a type of software testing that is performed to verify the non-functional requirements

of the application. It verifies whether the behavior of the system is as per the requirement or not. It tests all the aspects which are not tested in functional testing. Non-functional testing is defined as a type of software testing to check non-functional aspects of a software application. It is designed to test the readiness of a system as per nonfunctional parameters which are never addressed by functional testing. Non-functional testing is as important as functional testing.

Below is the difference between functional and non-functional testing:

Functional Testing	Non-functional Testing
It verifies the operations and actions of an application.	It verifies the behavior of an application.
It is based on requirements of customer.	It is based on expectations of customer.
It helps to enhance the behavior of the application.	It helps to improve the performance of the application.

Functional Testing	Non-functional Testing
Functional testing is easy to execute manually.	It is hard to execute non-functional testing manually.
It tests what the product does.	It describes how the product does.
Functional testing is based on the business requirement.	Non-functional testing is based on the performance requirement.
Examples: 1. Unit Testing 2. Smoke Testing 3. Integration Testing 4. Regression Testing	Examples: 1. Performance Testing 2. Load Testing 3. Stress Testing 4. Scalability Testing

- **Maintenance testing**

Maintenance testing is crucial post-release software testing that verifies changes (bug fixes, new features, environment updates) in a live system, primarily using regression testing to ensure new updates don't break

existing functionality, maintaining software stability and quality throughout its lifecycle. It covers corrective (bug fixes), adaptive (environment changes), perfective (enhancements), and preventive (performance/maintainability) activities, requiring impact analysis to scope the testing effort, say experts from ISTQB Foundation and LambdaTest.

Key Aspects

- **Triggers:** Updates, upgrades, migrations (e.g., to a new platform), retirement, or unplanned hotfixes.
- **Purpose:** Ensure changes work as intended and don't introduce negative side effects (regressions) in unchanged parts of the system.
- **Core Activity: Regression testing**, which re-tests existing features after modifications.
- **Process:** Involves analyzing the impact of the change, deciding on the scope, executing tests (often similar to pre-release), and verifying success.
- **Types of Changes:** Fixes (corrective), environmental adaptations (adaptive), performance improvements (perfective), or retirement data migration.

Why It's Important

- **Stability:** Prevents new issues from appearing in stable, working parts of the application.

- **User Experience:** Ensures a predictable and reliable experience after updates.
- **Longevity:** Keeps software healthy and functional as environments and requirements evolve.

Challenges

- Outdated documentation or missing specifications.
- Lack of domain knowledge among testers.
- Keeping automated tests updated with rapid code changes (test maintenance).

- **White Box Testing- Direct Inspections**

"Direct Inspections" in white box testing primarily refers to static testing methods such as code reviews, walkthroughs, and formal inspections (e.g., Fagan inspections), where the source code is manually examined without being executed. This is distinct from dynamic white box testing techniques that involve running the code.

- **White Box Testing and Direct Inspections**

White box testing, also known as clear box, glass box, or structural testing, requires the tester to have full knowledge of the software's internal workings, including source code, structure, and logic. Direct inspections are a core part of its static analysis methods.

The primary goal of these direct inspections is to find defects early in the Software Development Life

Cycle (SDLC), improve code quality, and ensure the code adheres to design specifications and coding standards.

Key Methods of Direct Inspection

- **Desk Checking:** The most basic form of static testing where the programmer manually reviews their own code before compilation or execution. The code is compared against requirements to ensure compliance and identify immediate errors.
- **Code Walkthroughs:** A semi-formal review technique where a group of technical personnel (e.g., peers, technical leads) go through the code. The author explains the logic while others ask questions and identify potential issues.
- **Code Inspections (e.g., Fagan Inspection):** A formal and structured process with specific phases (planning, overview, preparation, inspection meeting, rework, follow-up) aimed at detecting all faults and violations of coding standards.
- **Static Analysis Tools:** Automated tools (e.g., SonarQube, Checkmarx) are used to scan the source code without execution to identify potential issues such as syntax errors, security vulnerabilities, code complexity, and areas that do not adhere to coding standards.

Focus Areas of Direct Inspections

During these direct inspections, reviewers focus on various aspects of the code:

- **Code Logic and Flow:** Verifying that control structures (if-else statements, loops) and logical paths operate as intended.
- **Data Flow and Variables:** Ensuring variables are properly initialized, used, and updated to prevent data-related anomalies.
- **Error Handling:** Checking that exception handling mechanisms are robust and provide clear feedback.
- **Security Vulnerabilities:** Examining code for potential security flaws like injection flaws, buffer overflows, or weak authentication procedures.
- **Adherence to Standards:** Confirming the code follows organizational coding guidelines and best practices.

By performing these direct inspections, teams can identify fundamental issues that might be missed by dynamic testing, leading to cleaner, more maintainable, and robust software.

- **Walkthroughs:**

In software testing, a **walkthrough** is an informal peer review where the author of a document or code guides team members through it, explaining the logic and gathering feedback to find errors and improve understanding, often for high-level items like requirements or designs, focusing on knowledge sharing and consensus rather than formal defect correction. It's a type of static testing, performed without executing the code, with the goal of early

defect detection and quality enhancement, acting as an open-ended discussion led by the creator.

Key Characteristics

- **Led by Author:** The person who created the work product (document, code, test plan) leads the session.
- **Informal & Open:** Generally less formal than inspections, allowing for broad discussion and diverse perspectives.
- **Knowledge Sharing:** Excellent for getting stakeholders (even non-technical ones) up to speed on a product or feature.
- **Focus on Understanding:** Aims to achieve a common understanding and gather feedback on content, requirements, or logic.
- **Early Defect Detection:** Identifies issues like errors, ambiguities, or standard violations before execution.
- **Static Technique:** Performed on work products (like SRS, design docs, code) without running the actual software.

What is a Software Technical Review?

A **software technical review** is a formal process where software artifacts such as requirements, design documents, source code, test plans, and other related documents are examined by a team of peers to identify defects and improvement opportunities. The primary goal is to ensure that the software meets its specified

requirements and is free from defects before moving on to the next phase of development.

1. **Defect Identification:** To detect defects and issues early in the software development lifecycle.
2. **Quality Assurance:** To ensure that the software meets the quality standards and requirements.
3. **Improvement:** To suggest improvements in design, code, and documentation.
4. **Knowledge Sharing:** To facilitate the exchange of ideas and knowledge among team members.
5. **Compliance:** To ensure that the software adheres to regulatory and compliance standards.

- **Types Software Technical Review**

The intent of this section is for the students to realize that the types of reviews that must be performed on a project are dependent upon the specific intermediate deliverables that are produced. Various application areas should also be described in the context of their review processes.

1. Peer Reviews

Informal reviews conducted by colleagues to provide quick feedback and suggestions.

2. Walkthroughs

Semi-formal reviews where the author of the document or code explains the content to a group of reviewers.

3. Inspections

Formal reviews with a structured process and predefined roles, focusing on defect detection and correction.

4. Audits

Reviews conducted to ensure compliance with standards, regulations, and contractual agreements.

Difference between Formal and Informal Reviews

Aspects	Formal reviews	Informal reviews
Purpose and Objectives	- Detect defects - Ensure compliance with standards - Verify software requirements	- Provide quick feedback - Catch obvious defects - Share knowledge
Process and Structure	Follows a structured process - Includes planning, preparation, review meeting,	Flexible and unstructured process - May involve ad-hoc or impromptu reviews

Aspects	Formal reviews	Informal reviews
	rework, and follow-up	
Documentation	Extensive documentation - Formal records of defects and issues - Meeting minutes and action items	Minimal to no documentation - May rely on verbal feedback or brief notes
Participants	Includes specific roles such as moderator, author, reviewers, and scribe - Participants are pre-selected	Can include any relevant stakeholders - Participants are usually chosen informally

Functional Testing Process

The process of conducting functional testing typically involves several systematic steps:

1. **Understand Requirements:** Thoroughly analyze the functional requirements and user stories to define what the software is expected to do.

2. **Identify Test Inputs:** Determine the necessary input data and scenarios (both valid and invalid) to test the identified functions.
3. **Compute Expected Outcomes:** Define the anticipated result for each test case based on the software's specifications.
4. **Execute Test Cases:** Run the test cases using the prepared input data, either manually or through automation tools.
5. **Compare Actual vs. Expected Outcomes:** Compare the actual output of the system with the expected results. Any discrepancy is logged as a defect.
6. **Report and Retest:** Log defects with detailed steps to reproduce, and retest the functionality after developers apply a fix.

Types of Functional Testing

Functional testing encompasses several types of testing performed at different stages of the software development lifecycle:

- **Unit Testing:** Tests individual components or the smallest units of code, typically performed by developers early in the process.
- **Integration Testing:** Verifies that different modules or units work correctly when combined and interact with each other.

- **System Testing:** Evaluates the complete and fully integrated software system to ensure it meets all specified requirements.
- **User Acceptance Testing (UAT):** The final phase where end-users or clients test the software to confirm it meets their business needs before final deployment.
- **Smoke Testing:** A quick, preliminary test on a new build to ensure that the most critical functions are working and the application is stable enough for further testing.
- **Sanity Testing:** A subset of regression testing to ensure that specific changes or bug fixes work as expected and haven't introduced new issues to other functions.
- **Regression Testing:** Involves re-running tests to ensure that new code changes do not negatively affect existing functionalities.

- **Basis path testing**

Basis path testing is a white-box testing technique that uses the program's control structure to ensure every independent path in the code's control flow is executed at least once. The goal is to achieve thorough code coverage, including all logical conditions and branches, to detect logical errors.

Key Concepts

- **White-Box Testing:** This method requires the tester to have full knowledge and access to the source code and its internal structure, as opposed to black-box testing, which focuses on the external behavior.
- **Control Flow Graph (CFG):** The program's logic is visually represented as a directed graph, where nodes represent code statements/blocks and edges show the flow of control.
- **Independent Path:** A specific route through the CFG that introduces at least one new edge not traversed by any other independent path identified so far.
- **Cyclomatic Complexity:** A metric that quantifies the logical complexity of a program and determines the *minimum* number of independent paths needed for complete branch coverage. It serves as an upper bound for the number of test cases required.

Steps in Basis Path Testing

The process generally involves four main steps:

1. **Construct the Control Flow Graph (CFG):** The source code is translated into a graph format, identifying nodes (statements/blocks) and edges (control flow).
2. **Calculate the Cyclomatic Complexity:** The complexity is computed using one of several formulas, most commonly:

◦

$V(G)=E-N+2P$ cap V open paren cap G close paren equals cap E minus cap N plus 2 cap P

$$V(G)=E-N+2P$$

(where E is the number of edges, N is the number of nodes, and P is the number of connected components, typically 1 for a single function).

o

$V(G)=\text{Number of predicate nodes}+1$ cap V open paren cap G close paren equals Number of predicate nodes plus 1

$$V(G)=\text{Number of predicate nodes}+1$$

(predicate nodes are decision points with more than one exit edge).

o

$V(G)=\text{Number of regions}$ cap V open paren cap G close paren equals Number of regions

$$V(G)=\text{Number of regions}$$

(areas bounded by edges and nodes in the graph).

3. Identify Independent Paths: A basis set of unique, independent paths is identified, the count of which matches the calculated cyclomatic complexity.
4. Design Test Cases: Specific test cases with appropriate input data are created to force the

execution of each independent path identified in the basis set.

- **Control Structure Testing**

Control structure testing is used to increase the coverage area by testing various control structures present in the program.

1. Condition Testing
2. Data Flow Testing
3. Loop Testing

1. Condition Testing : Condition testing is a test cased design method, which ensures that the logical condition and decision statements are free from errors. The errors present in logical conditions can be incorrect boolean operators, missing parenthesis in a booleans expression, error in relational operators, arithmetic expressions, and so on. The common types of logical conditions that are tested using condition testing are-

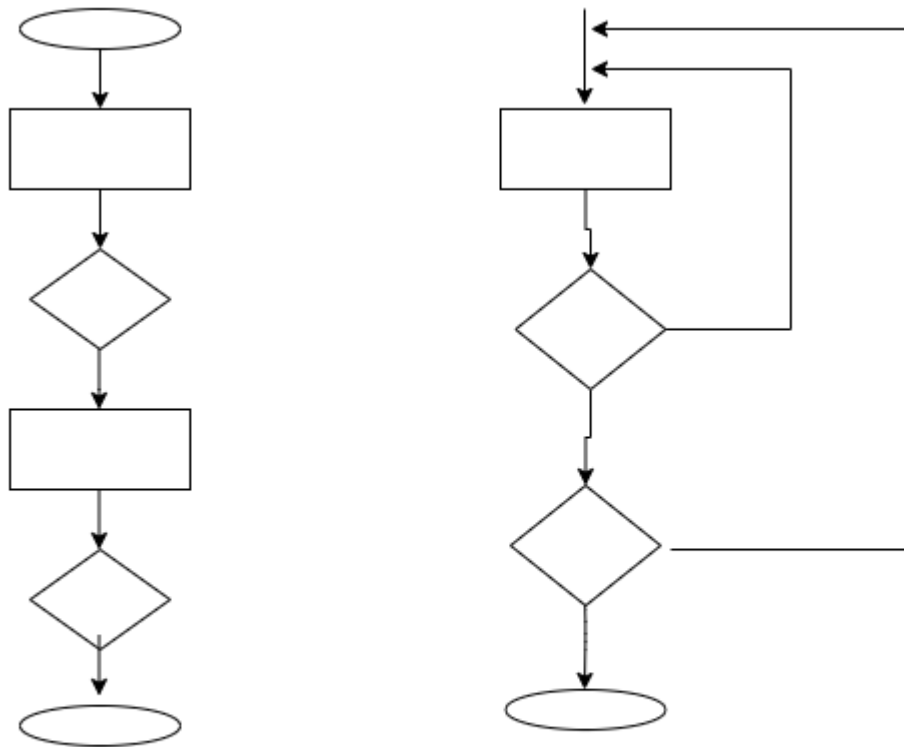
2. Data Flow Testing : The data flow test method chooses the test path of a program based on the locations of the definitions and uses all the variables in the program. The data flow test approach is depicted as follows suppose each statement in a program is assigned a unique statement

number and that theme function cannot modify its parameters or global variables.

3. Loop Testing : Loop testing is actually a white box testing technique. It specifically focuses on the validity of loop construction. Following are the types of loops.

1. **Simple Loop** - The following set of test can be applied to simple loops, where the maximum allowable number through the loop is n .
 1. Skip the entire loop.
 2. Traverse the loop only once.
 3. Traverse the loop two times.
 4. Make p passes through the loop where $p < n$.
 5. Traverse the loop $n-1$, n , $n+1$ times.
2. **Concatenated Loops** - If loops are not dependent on each other, contact loops can be tested using the approach used in simple loops. if the loops are interdependent, the steps are followed in nested

loops.



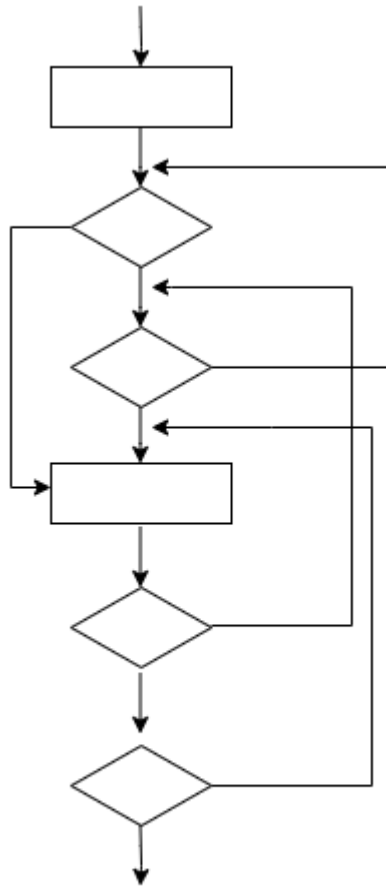
Concatenated Loops

3. **Nested Loops** - Loops within loops are called as nested loops. when testing nested loops, the number of tested increases as level nesting increases. The following steps for testing nested loops are as follows-

1. Start with inner loop. set all other loops to minimum values.
2. Conduct simple loop testing on inner loop.
3. Work outwards.
4. Continue until all loops tested.

4. **Unstructured loops** - This type of loops should be redesigned, whenever possible, to reflect the use of

unstructured the structured programming



constructs. **Unstructured Loops**

- **Code Coverage Testing**

Code Coverage Testing is a white-box testing metric that measures the percentage of your source code executed during automated tests. It helps developers identify untested areas, improve test quality, and gain confidence in the software's reliability, though high coverage alone does not guarantee a bug-free product.

Importance and Benefits

- **Identifies Untested Code:** It highlights code paths, branches, or functions that have not been exercised by tests, guiding the creation of new test cases.
- **Improves Quality:** Striving for good coverage encourages more comprehensive testing, which helps detect bugs earlier in the development lifecycle, reducing the cost and effort of fixing them later.
- **Facilitates Refactoring:** Knowing that existing code is covered by tests gives developers the confidence to refactor and optimize the codebase without the fear of breaking existing functionality (regressions).
- **Aids in Compliance:** In certain safety-critical industries (e.g., avionics, automotive), specific levels of code coverage are mandated by functional safety standards.
- **Identifies Dead Code:** It helps pinpoint "dead" or unreachable code that can be removed, making the codebase cleaner and easier to maintain.

Types of Code Coverage Metrics

Different metrics provide various levels of detail about the code execution.

- **Line Coverage (or Statement Coverage):** Measures the percentage of executable lines (or statements) that have been run at least once. It is the most basic metric.

- Branch Coverage (or Decision Coverage): Ensures that both the "true" and "false" branches of conditional statements (e.g., if-else or switch statements) are executed.
- Function Coverage: Tracks whether each function or method in the source code has been called during testing.
- Condition Coverage: Verifies that each individual boolean sub-expression within a conditional statement has been evaluated to both true and false.
- Path Coverage: The most rigorous and complex metric, ensuring all possible execution paths through the code are tested. This can be impractical for large applications due to the exponential number of paths.

Common Code Coverage Tools

Tools automate the process of measuring coverage and generating reports, often integrating with build and CI/CD pipelines.

- Java: JaCoCo, Cobertura, Clover
- Python: Coverage.py
- JavaScript/Node.js: Istanbul (nyc)
- .NET/C#: dotCover, NCover

• **Code Complexity Testing**

Code complexity testing is the process of quantitatively assessing the difficulty of understanding, testing, and

maintaining software code using specific metrics and testing methods. This is a crucial part of white-box testing, helping to identify high-risk areas prone to defects, which then guides testing efforts and potential refactoring.

Key Metrics for Code Complexity Testing

Software metrics provide objective measures for complexity. The most common ones are:

- **Cyclomatic Complexity:** Measures the number of linearly independent paths through a program's source code, often by counting decision points (e.g., if, for, while, switch statements).
 - **Use in Testing:** This metric directly indicates the minimum number of test cases required to achieve full path coverage. A high score (e.g., above 10-15) suggests high risk and difficulty in testing all scenarios.
- **Cognitive Complexity:** A more human-centric metric that measures the mental effort required for a developer to understand the code. It penalizes factors like deep nesting and recursion more heavily than structural metrics.
 - **Use in Testing:** Highlights code that is hard for humans to reason about, which makes it more likely for developers to introduce errors during modifications and for testers to miss edge cases.

- **Maintainability Index (MI):** A composite score typically combining cyclomatic complexity, Halstead volume, and lines of code (LoC). It provides an overall view of how easy the code is to maintain.
 - **Use in Testing:** A low MI score serves as a strong signal that the module needs refactoring to reduce future maintenance costs and testing effort.
- **Halstead Complexity Measures:** Analyze the code's structure and vocabulary by counting operators and operands. This helps estimate the effort required to write or understand the code.
- **Lines of Code (LoC):** A basic measure of program size. While not a complete picture of complexity on its own, when combined with other metrics, it can help identify functions that are too large and unwieldy.

Role in Software Testing

Code complexity testing is used to guide the overall testing strategy:

- **Risk Assessment:** Modules with high complexity scores are identified as high-risk areas and prioritized for more rigorous testing and code reviews.

- **Test Case Design:** Complexity metrics, especially cyclomatic complexity, help determine the necessary number of test cases to ensure adequate code coverage.
- **Resource Allocation:** Testers and QA managers can use these metrics to allocate more time and resources to testing complex components.
- **Informing Refactoring:** The results of complexity analysis provide objective data points that help developers decide which parts of the codebase need simplification or modularization to improve testability and reduce bugs.

Tools and Practices

Complexity testing often involves automated static analysis tools that integrate into the development workflow and CI/CD pipelines. Popular tools include SonarQube, PMD, and various IDE plugins that provide real-time feedback. Regular code reviews also help identify and manage complexity qualitatively.

Black Box Testing- Requirement Based Testing

Black box testing is a software testing methodology that validates an application's functionality from an external, user-focused perspective, without knowledge of its internal code or structure. **Requirement-based testing** is a specific approach within black box testing

that systematically verifies that each specified functional and non-functional requirement is met by the software.

Black Box Testing

Black box testing, also known as behavioral, functional, or specification-based testing, focuses entirely on the inputs and outputs of the software system.

- **Core Principle:** The software is treated as an opaque "black box." The tester does not need programming skills or access to the source code; instead, they interact with the application's user interface and provide various inputs to observe if the outputs match expected results based on the documented requirements.
- **User-Centric Focus:** This method ensures the software meets end-user expectations and real-world usage scenarios, identifying issues with functionality, usability, and performance from the customer's point of view.
- **Techniques:** Common techniques used to design effective black box tests include:
 - **Equivalence Partitioning:** Dividing input data into valid and invalid classes and testing a representative value from each.
 - **Boundary Value Analysis (BVA):** Focusing on testing values at the edges of valid input ranges,

where errors are more likely to occur (e.g., for a valid range of 10-100, testing 9, 10, 100, and 101).

- **Decision Table Testing:** Creating a table to test combinations of conditions and their corresponding actions, useful for complex business rules.
- **State Transition Testing:** Validating that the system correctly transitions between different states based on specific inputs or events.
- **Error Guessing:** Using a tester's experience and intuition to anticipate where defects might occur and designing test cases for those scenarios.

Requirement-Based Testing

Requirement-based testing is a foundational element and key focus area of the black box approach.

- **Definition:** It is a testing process where test cases are derived directly from the software's functional and non-functional requirements and specifications documents.
- **Purpose:** The primary goal is to guarantee that every documented feature and behavior outlined during the initial phases of the project has been implemented correctly and functions as expected.

- **Process:**

- **Understand Requirements:** Testers thoroughly analyze the requirements or user stories to determine what the system should do.
- **Design Test Cases:** Test scenarios are created that directly map to each specific requirement.
- **Ensure Traceability:** Clear links are established between each test case and its corresponding requirement, allowing for easy tracking and ensuring all requirements are covered.
- **Execute and Validate:** Tests are executed, and the actual results are compared against the expected outcomes specified in the requirements.

In essence, requirement-based testing is how black box testing achieves its primary objective: ensuring the software meets the documented specifications and user needs.

What is Requirement-Based Testing in Software Development

Requirement-based testing in software development refers to a crucial process that involves validating a software system based on its specified criteria. This approach guarantees that the software

aligns with documented specifications and meets the anticipated outcomes outlined during the initial phases of the project. Testers generate test cases based on these requirements, methodically assessing each element to ensure proper functionality. Making testing strategies align with the requirements enhances the chances of delivering a software product that complies with predefined benchmarks and user expectations.

Principles of Requirement-Based Testing

1. **Traceability:** The approach aims to establish clear links between each test and its respective requirements, ensuring easy tracking.
2. **Early Engagement:** Early involvement in testing allows teams to comprehend, validate, and clarify requirements, minimizing misinterpretation risks.
3. **Validation and Verification:** This methodology focuses on both aspects to ensure software compliance with specified requirements, boosting testing reliability.
4. **Adaptability:** Flexibility is a crucial aspect, enabling adjustments in requirements without compromising testing objectives.
5. **Coverage:** Comprehensive testing coverage ensures systematic testing of all identified requirements throughout the testing phase.

6. Documentation: Accurate documentation of requirements, test cases and results captures the entire testing process for reference.

Significance of Requirement Based Testing in Software Development

1. Alignment with Design Specifications

This testing approach ensures that the software aligns precisely with the functionalities and specifications outlined during the initial design phase. By testing against these specifications, it validates that the software behaves as intended.

2. Verification of Documented Requirements

This process ensures that the developed software accurately reflects the documented requirements. It's a critical step in ensuring that the software is constructed according to the defined criteria and expected standards.

3. Systematic Feature Evaluation

Through Requirement-Based Testing, software teams methodically evaluate each implemented feature against predetermined criteria. This thorough assessment ensures that every element functions as intended and meets the specified requirements.

4. Inconsistency Identification

This testing method plays a crucial role in detecting any disparities or deviations from the specified requirements.

Recognizing these discrepancies at an early development stage enables teams to tackle and rectify them, before they escalate into significant issues.

5. Improvement in the reliability and overall quality

Requirement Based Testing plays a major role in enhancement of the overall quality, features and dependability of the software. By ensuring that documented requirements are aligned, it boosts the software's functionality, performance and user satisfaction, leading to a more reliable end product.

Types of Requirements in Software Development

Understanding software requirements and its different aspect is done briefly below

- 1. Functional Requirements:** They detail specific tasks, calculations, data handling and built-in features.
- 2. Non-Functional Requirements:** They cover attributes like performance, speed, user-friendliness, security and reliability.
- 3. Business Requirements:** They define the primary goals and desired outcomes, making sure the software serves a higher purpose.
- 4. User Requirements:** They consider user preferences, usability criteria, and accessibility concerns.

5. **System Requirements:** These define the essential environment and infrastructure the software needs to run smoothly. They specify hardware compatibility, software dependencies, network configurations, and other technical parameters.
6. **Stakeholder Requirements:** These gather valuable input from all parties involved in the software's development and use, including customers, users, developers, and even regulatory entities. They ensure diverse perspectives are heard and addressed.
7. **Interface Requirements:** These define how the software interacts with external systems and platforms. They cover communication protocols, data formats, and API specifications.

Boundary Value Analysis

Boundary Value Analysis is based on testing the boundary values of valid and invalid partitions. The behavior at the edge of the equivalence partition is more likely to be incorrect than the behavior within the partition, so boundaries are an area where testing is likely to yield defects.

It checks for the input values near the boundary that have a higher chance of error. Every partition has its maximum and minimum values and these maximum and minimum values are the boundary values of a partition.

In simple terms boundary value Analysis is like testing the edge cases of our software where most of the time it will get broke so it is important to do BVA before deploying the code. There are many other test that are done if you wish to learn them all then you can join our [complete software testing course](#)

Note:

- A boundary value for a valid partition is a valid boundary value.
- A boundary value for an invalid partition is an invalid boundary value.
- For each variable we check-
 - Minimum value.
 - Just above the minimum.
 - Nominal Value.
 - Just below Max value.
 - Max value.

Example: Consider a system that accepts ages from 18 to 56.

Boundary Value Analysis(Age accepts 18 to 56)		
Invalid	Valid	Invalid

Boundary Value Analysis(Age accepts 18 to 56)

(min-1)	(min, min + 1, nominal, max - 1, max)	(max + 1)
17	18, 19, 37, 55, 56	57

Valid Test cases: Valid test cases for the above can be any value entered greater than 17 and less than 57.

- Enter the value- 18.
- Enter the value- 19.
- Enter the value- 37.
- Enter the value- 55.
- Enter the value- 56.

Invalid Testcases: When any value less than 18 and greater than 56 is entered.

- Enter the value- 17.
- Enter the value- 57.

Single Fault Assumption: When more than one variable for the same application is checked then one can use a single fault assumption. Holding all but one variable to the extreme value and allowing the remaining variable to take the extreme value. For n variable to be checked:

Maximum of $4n+1$ test cases

Problem: Consider a Program for determining the Previous Date.

Input: *Day, Month, Year with valid ranges as-*

$$1 \leq \text{Month} \leq 12$$

$$1 \leq \text{Day} \leq 31$$

$$1900 \leq \text{Year} \leq 2000$$

Design Boundary Value Test Cases.

Solution: Taking the year as a Single Fault Assumption i.e. year will be having values varying from 1900 to 2000 and others will have nominal values.

Test Cases	Month	Day	Year	Output
1	6	15	1900	14 June 1900
2	6	15	1901	14 June 1901
3	6	15	1960	14 June 1960
4	6	15	1999	14 June 1999
5	6	15	2000	14 June 2000

Taking Day as Single Fault Assumption i.e. Day will be having values varying from 1 to 31 and others will have nominal values.

Test Case	Month	Day	Year	Output
6	6	1	1960	31 May 1960
7	6	2	1960	1 June 1960
8	6	30	1960	29 June 1960
9	6	31	1960	Invalid day

Taking Month as Single Fault Assumption i.e. Month will be having values varying from 1 to 12 and others will have nominal values.

Test Case	Month	Day	Year	Output
10	1	15	1960	14 Jan 1960
11	2	15	1960	14 Feb 1960
12	11	15	1960	14 Nov 1960
13	12	15	1960	14 Dec 1960

For the n variable to be checked Maximum of $4n + 1$ test case will be required. Therefore, for $n = 3$, the maximum test cases are-

$$4 \times 3 + 1 = 13$$

- **Equivalence Partitioning**

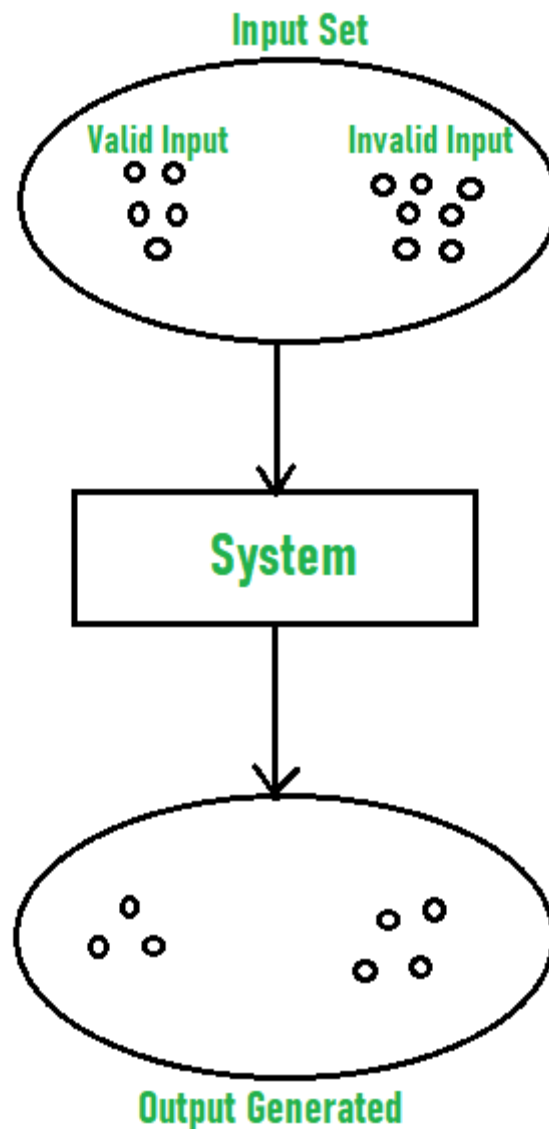
Equivalence Partitioning Method is also known as Equivalence class partitioning (ECP). It is a software testing technique or black-box testing that divides input domain into classes of data, and with the help of these classes of data, test cases can be derived. An ideal test case identifies class of error that might require many arbitrary test cases to be executed before general error is observed.

In equivalence partitioning, equivalence classes are evaluated for given input conditions. Whenever any input is given, then type of input condition is checked, then for this input conditions, Equivalence class represents or describes set of valid or invalid states.

Guidelines for Equivalence Partitioning :

- If the range condition is given as an input, then one valid and two invalid equivalence classes are defined.
- If a specific value is given as input, then one valid and two invalid equivalence classes are defined.
- If a member of set is given as an input, then one valid and one invalid equivalence class is defined.

- If Boolean no. is given as an input condition, then one valid and one invalid equivalence class is defined.



Example-1:

Let us consider an example of any college admission process. There is a college that gives admissions to students based upon their percentage.

Consider percentage field that will accept percentage

only between 50 to 90 %, more and even less than not be accepted, and application will redirect user to an error page. If percentage entered by user is less than 50 % or more than 90 %, that equivalence partitioning method will show an invalid percentage. If percentage entered is between 50 to 90 %, then equivalence partitioning method will show valid percentage.

Percentage * Accepts Percentage value between 50 to 90

Equivalence Partitioning		
Invalid	Valid	Invalid
≤ 50	50-90	≥ 90

Example 2:

Let us consider an example of an online shopping site. In this site, each of products has a specific product ID and product name. We can search for product either by using name of product or by product ID. Here, we consider search field that accepts only valid product ID or product name.

Let us consider a set of products with product IDs and

users wants to search for Mobiles. Below is a table of some products with their product Id.

Product	Product ID
Mobiles	45
Laptops	54
Pen Drives	67
Keyboard	76
Headphones	34

If the product ID entered by user is invalid then application will redirect customer or user to error page. If product ID entered by user is valid i.e. 45 for mobile, then equivalence partitioning method will show a valid product ID.

Search

Equivalence Partioning		
Invalid	Invalid	Valid
77	84	45

Example-3

Let us consider an example of software application. There is function of software application that accepts only particular number of digits, not even greater or less than that particular number.

Consider an OTP number that contains only 6 digit number, greater and even less than six digits will not be accepted, and the application will redirect customer or user to error page. If password entered by user is less or more than six characters, that equivalence partitioning method will show an invalid OTP. If password entered is exactly six characters, then equivalence partitioning method will show valid OTP.

Enter OTP

*Must include six digits

Equivalence Partioning			
Invalid	Invalid	Valid	Valid
Digits>=7	Digits<=5	Digits=6	Digits=6
67545678	9754	654757	213309

• Differences between BBT and WBT

Black Box Testing Vs White Box Testing

Parameters	Black Box Testing	White Box Testing
Definition	Black Box Testing is a way of software testing in which the internal structure or the program or the code is hidden and nothing is known about it.	White Box Testing is a way of testing the software in which the tester has knowledge about the internal structure or the code or the program of the software.
Testing objectives	Black box testing is mainly focused on testing the functionality of the software, ensuring that it meets the requirements and specifications.	White box testing is mainly focused on ensuring that the internal code of the software is correct and efficient.
Testing methods	Black box testing uses methods like Equivalence	White box testing uses methods like Control Flow

Parameters	Black Box Testing	White Box Testing
	Partitioning, Boundary Value Analysis, and Error Guessing to create test cases.	Testing, Data Flow Testing and Statement Coverage Testing.
Knowledge level	Black box testing does not require any knowledge of the internal workings of the software, and can be performed by testers who are not familiar with programming languages.	White box testing requires knowledge of programming languages, software architecture and design patterns.
Scope	Black box testing is generally used for testing the software	White box testing is used for testing the software at the unit level,

Parameters	Black Box Testing	White Box Testing
	at the functional level.	integration level and system level.
Implementation	Implementation of code is not needed for black box testing.	Code implementation is necessary for white box testing.
Done By	Black Box Testing is mostly done by software testers.	White Box Testing is mostly done by software developers.
Terminology	Black Box Testing can be referred to as outer or external software testing.	White Box Testing is the inner or the internal software testing.
Testing Level	Black Box Testing is a functional test of the software.	White Box Testing is a structural test of the software.

Parameters	Black Box Testing	White Box Testing
Testing Initiation	Black Box testing can be initiated based on the requirement specifications document.	White Box testing of software is started after a detail design document.
Programming	No knowledge of programming is required.	It is mandatory to have knowledge of programming.
Testing Focus	Black Box Testing is the behavior testing of the software.	White Box Testing is the logic testing of the software.
Applicability	Black Box Testing is applicable to the higher levels of testing of software.	White Box Testing is generally applicable to the lower levels of software testing.

Parameters	Black Box Testing	White Box Testing
Alternative Names	Black Box Testing is also called closed testing.	White Box Testing is also called as clear box testing.
Time Consumption	Black Box Testing is least time consuming.	White Box Testing is most time consuming.
Suitable for Algorithm Testing	Black Box Testing is not suitable or preferred for algorithm testing.	White Box Testing is suitable for algorithm testing.
Approach	Can be done by trial and error ways and methods.	Data domains along with inner or internal boundaries can be better tested.
Example	Search something on google by using keywords	By input to check and verify loops

Parameters	Black Box Testing	White Box Testing
Exhaustiveness	It is less exhaustive as compared to white box testing.	It is comparatively more exhaustive than black box testing.